BOLSA DE TRABAJO IEN

Documentación del proyecto
y modelo de datos

Revisión del código local · 22 de octubre de 2026

21 entidades propias · 212 campos locales

Siete componentes según la guía de documentación

Incluye requisitos, arquitectura, decisiones técnicas, contratos de integración, diccionario de datos, despliegue y calidad. Distingue implementación observada, procedimientos propuestos y verificaciones pendientes.

Base documental: Guia_documentacion.pdf y repositorio BolsaTrabajo. No incluye datos personales, credenciales ni resultados de pruebas no ejecutadas.

Contenido

01 · Guía y alcance

02 · Requisitos y aceptación

03 · Arquitectura

04 · Decisiones técnicas: persistencia

05 · Decisiones técnicas: integración

06 · Contratos HTTP y WebSocket

07 · Modelo de datos y diccionario

08 · Despliegue y operación

09 · Pruebas y calidad

10 · Revisión documental

Documentación de Bolsa de Trabajo IEN

Fecha de revisión: 22/10/2026. Basada en Guia_documentacion.pdf y en el código local de BolsaTrabajo. Incluye cambios locales todavía no confirmados en Git. Esta entrega documenta el sistema; no acredita un despliegue ni una ejecución integral de pruebas.

Producto y alcance

Bolsa de Trabajo IEN conecta postulantes con empresas y vincula las brechas de competencias con cursos. Permite registrar cuentas, validar empresas, moderar ofertas, administrar catálogos, guardar favoritas, gestionar CV privados y consultar análisis y compatibilidad mediante un servicio externo de IA.

Los actores son visitante, postulante, empresa y personal de moderación/administración. Los perfiles Postulante y Oferente se vinculan con Usuario. Los permisos efectivos dependen de decoradores, formularios y vistas; la base no impone exclusión entre ambos perfiles.

No existe una entidad de postulación laboral ni un proceso persistido de selección/contratación en los modelos actuales. Guardar una favorita o calcular compatibilidad no registra una candidatura.

Los siete componentes de la guía

N.º	Componente	Entrega y evidencia	Responsable sugerido
01	Inicio y configuración	README, dependencias y .env.example	Responsable técnico
02	Requisitos y aceptación	Backlog funcional	Referente funcional + responsable de módulo
03	Arquitectura y decisiones	Arquitectura y ADR	Responsable técnico
04	Contratos de integración	HTTP, callbacks y WebSocket	Responsables Django/FastAPI
05	Modelo de datos	Modelo y diccionario, ER editable	Responsable de datos
06	Despliegue y operación	Procedimientos	Infraestructura + desarrollo
07	Pruebas y calidad	Estrategia	Equipo de desarrollo

Estado y límites de la entrega

- El esquema documentado contiene 21 entidades propias; las tablas técnicas de Django y las tablas puente se aclaran por separado.

- Los contratos del servicio de IA se describen desde sus consumidores en este repositorio. La implementación FastAPI, Celery y sus OpenAPI no están disponibles en esta carpeta y deben contrastarse antes de integrar una versión nueva.

- Los procedimientos productivos son una propuesta reproducible: no se encontró configuración de hosting, unidades systemd ni un pipeline CI de pruebas en .github.

- La retención, los objetivos de recuperación y los responsables personales todavía requieren definición institucional. No se inventaron plazos ni resultados de pruebas.

Mantenimiento

Actualizar README y operación al cambiar comandos o configuración; requisitos al modificar el comportamiento; arquitectura y ADR al alterar límites o protocolos; contratos ante cualquier cambio observable; modelo y migraciones ante cambios del esquema; pruebas cuando cambian riesgos o rutas críticas. Cada PR debe actualizar los componentes afectados. El backlog de esta entrega es el índice funcional inicial; los identificadores RF deben reutilizarse al vincular tareas o issues reales.

Fuentes de implementación

Modelos y migraciones de apps/{usuarios,categorias,ofertas,cursos,ia,matching}; rutas y vistas; ientrabajo/settings/base.py y ientrabajo/asgi.py; contratos Pydantic de ien_contracts/models.py; comandos de despacho; requirements.txt; package.json; documentación técnica existente. La guía adjunta se usó como estructura de la entrega, sin ejecutar instrucciones ajenas a la solicitud del usuario.

Requisitos y criterios de aceptación

Revisión: 22/10/2026. Backlog derivado de código existente; no equivale a pruebas ejecutadas. P0 indica seguridad/integridad crítica; P1, funcionalidad central; P2, complemento. Los RF son identificadores locales y aún no tienen issues vinculados.

ID / prioridad	Historia y reglas	Criterios verificables	Evidencia / tarea de mantenimiento
RF-01 / P1	Como persona quiero registrarme y entrar con email. Email único; contraseña gestionada por Django.	Un email duplicado no crea otra cuenta; credenciales válidas abren sesión; credenciales inválidas no autentican.	usuarios/models.py, forms.py, views.py; mantener autenticación.
RF-02 / P0	Como moderador quiero aprobar o rechazar empresas. Estado inicial pendiente.	Una empresa pendiente no accede a publicar; un miembro staff aprueba/rechaza; otro rol no modera; al reenviar revisión se limpia el rechazo.	usuarios/estado_oferente.py, decorators.py, moderacion/views.py.
RF-03 / P1	Como empresa aprobada quiero crear y administrar mis ofertas.	La creación queda pendiente; editar exige propiedad y estado permitido; aprobar lleva a activa; finalizar impide tratarla como activa; rechazo conserva motivo.	ofertas/views.py, forms.py, state.py.
RF-04 / P1	Como visitante/postulante quiero consultar ofertas y cursos.	La búsqueda respeta los filtros definidos en la vista; el catálogo de cursos muestra activos; el acceso directo respeta la política de visibilidad de la vista.	ofertas/services.py, cursos/views.py; verificar listado y detalle.
RF-05 / P1	Como postulante quiero marcar ofertas favoritas.	Dos altas de la misma pareja no duplican filas; desmarcar elimina la relación; otra cuenta no modifica mis favoritas.	ofertas/models.py, services.py, views.py.
RF-06 / P0	Como postulante quiero cargar y descargar un CV privado.	Se rechazan archivos mayores a 5 MiB o de contenido inválido; se aceptan PDF, DOCX, ODT y TXT válidos; DOC/RTF dependen del conversor aislado; cada carga aceptada tiene número, hash y archivo privado; otra cuenta/anónimo no descarga; rutas públicas heredadas devuelven 404.	usuarios/uploads.py, services.py, views.py, ia/models.py; validar formatos habilitados.
RF-07 / P0	Como postulante quiero analizar mi CV sin esperar en la petición web.	Sin versión actual responde 400; solicitud válida responde 202 y crea/reutiliza el trabajo; el despachador entrega el snapshot; callback válido actualiza el análisis; firma inválida responde 401; evento duplicado no vuelve a aplicar cambios.	ia/views.py, workflow.py, comandos; verificar con dobles del proveedor.
RF-08 / P0	Como postulante quiero editar mi perfil sin usar información obsoleta en el match.	Se conservan versiones; la normalización debe finalizar antes del cálculo cuando corresponda; un resultado de CV anterior no se presenta como vigente; el conflicto devuelve 409.	ia/workflow.py, forms.py, matching/services.py; revisar nombres de formularios al modificar.
RF-09 / P1	Como postulante quiero ver compatibilidad y sus fundamentos.	Solo se calcula sobre oferta activa y perfiles aptos; la respuesta es 202 al encolar o 200 al reutilizar completado; resultado guarda hashes, versiones, pesos y snapshots; el detalle muestra requisitos y brechas; otra cuenta recibe 404 al consultar el UUID ajeno.	matching/views.py, services.py, ien_contracts/models.py.
RF-10 / P1	Como postulante quiero cancelar un cálculo pendiente.	Cancelar mi trabajo devuelve estado cancelled; un callback tardío no lo convierte en completado; se registra el evento de cancelación remota; cancelar el UUID de otra persona no lo modifica.	matching/views.py, services.py, dispatch_match_jobs.py.

ID / prioridad	Historia y reglas	Criterios verificables	Evidencia / tarea de mantenimiento
RF-11 / P1	Como administrador quiero publicar ponderaciones reproducibles.	Cada peso está entre 0 y 100; publicación exige suma exacta 100; una configuración publicada no se edita con las operaciones del modelo; los resultados conservan la configuración usada.	matching/models.py, admin.py, configuration.py; validar la ruta efectiva de publicación.
RF-12 / P2	Como postulante quiero cursos relacionados con mis brechas.	Solo se recomiendan cursos aptos según el algoritmo local; se usan habilidades declaradas del curso y brechas del resultado; no se realiza otra llamada LLM.	matching/course_recommendations.py, cursos/models.py.
RF-13 / P0	Como usuario quiero recibir solo mis eventos.	WebSocket anónimo rechaza con 4401; match ajeno rechaza con 4404; CV firmado exige token y sesión correspondientes; si no hay canal en vivo puede consultarse estado HTTP.	ia/consumers.py, matching/consumers.py, asgi.py.
RF-14 / P1	Como moderador quiero mantener categorías, habilidades y cursos.	No se borra categoría referenciada; no se repite habilidad por nombre/categoría; un usuario sin permisos no administra catálogos.	categorias/models.py, cursos/models.py, moderacion/views.py.

Restricciones y excepciones

El servicio IA puede fallar o limitar solicitudes. Un timeout no demuestra que el trabajo no fue aceptado: se mantienen claves de idempotencia, eventos y estados de entrega. Redis es necesario para canales y caché; la persistencia de negocio está en la base relacional. La compatibilidad es una ayuda informativa y no registra una decisión de contratación.

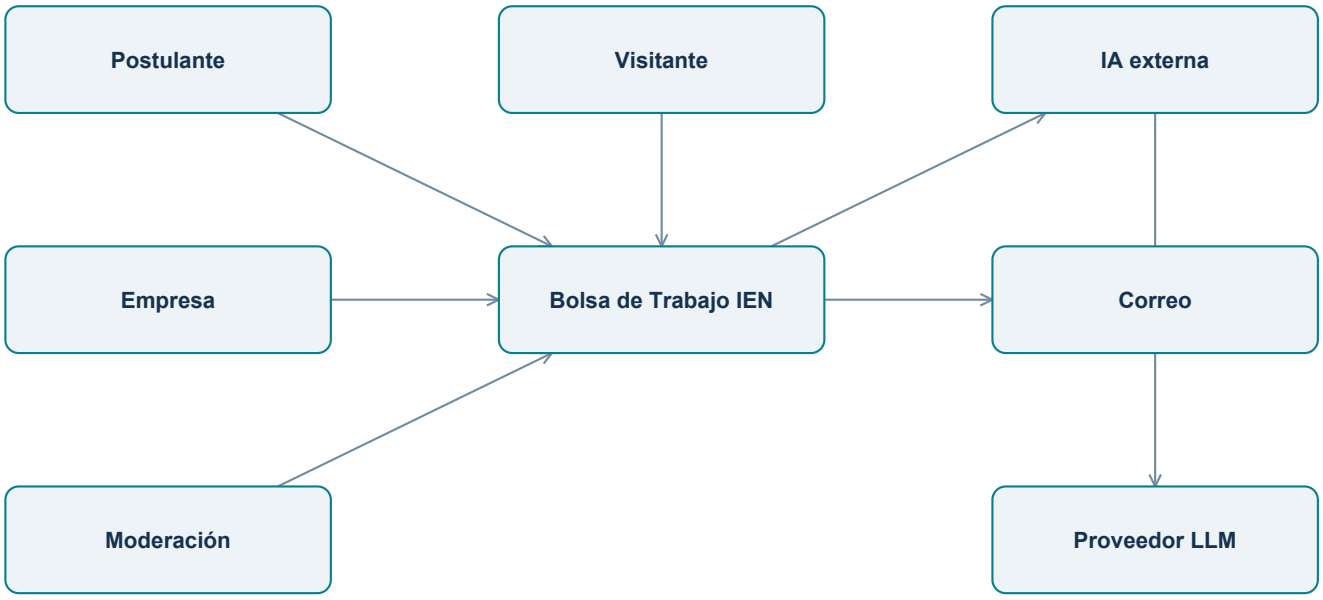
Pendientes funcionales para validar con el referente

Confirmar visibilidad esperada del detalle de ofertas/cursos inactivos, política de cuentas con ambos perfiles, tratamiento de CUIT repetido (hoy no es único en BD), reglas de conservación/borrado y objetivos de tiempo de respuesta. Crear issues reales para estos puntos y asociarles los RF; no se crearon tareas en servicios externos.

Arquitectura general

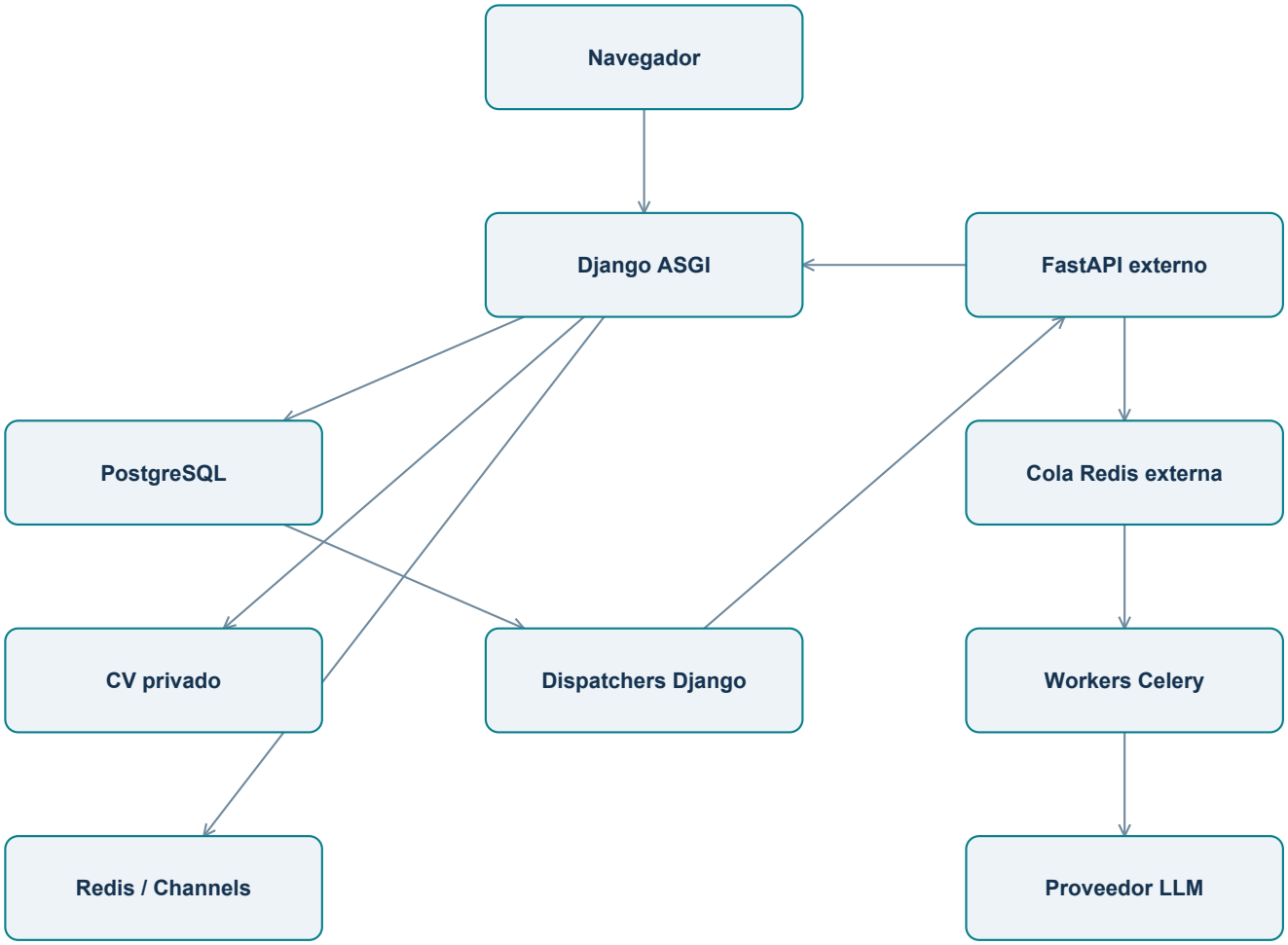
Revisión: 22/10/2026. Django usa MVT: modelos para persistencia, formularios para validación, servicios para casos de uso, vistas para HTTP y templates con JavaScript para interfaz. El stack declarado es Python, Django >=5.2,<6, Channels/Daphne, PostgreSQL/SQLite, Redis, Requests, WebSockets y Pydantic. La cabecera generada de algunos archivos menciona Django 6; la dependencia efectiva la define requirements.txt.

Contexto



Transporte y responsabilidades detallados en el texto y en el diagrama Mermaid editable.

Contenedores y límites



Transporte y responsabilidades detallados en el texto y en el diagrama Mermaid editable.

Redis/cola y Celery pertenecen al despliegue del servicio externo: aquí no hay aplicación Celery ni docker-compose propio. Los dos despachadores de Django son procesos separados de la web y deben iniciarse además del stack descrito en el README. La topología interna de FastAPI/Celery es referida por la documentación existente y debe verificarse en su repositorio.

Responsabilidad por módulo

Módulo	Responsabilidad
usuarios	Identidad por email, empresas/postulantes, carga y descarga de CV.
categorias	Catálogo compartido de categorías y competencias.
ofertas	Publicación, estados, búsqueda y favoritas.
cursos	Catálogo de formación y competencias desarrolladas.
moderacion	Operaciones staff sobre empresas, ofertas y catálogos; sin tabla propia actual.
emails	Servicios y plantillas de notificación; sin entidad propia actual.
ia	Versiones de CV, extracción/normalización, perfiles canónicos, outbox y recepción.
matching	Solicitudes de compatibilidad, rúbricas, caché, cancelación y resultados.
ien_contracts	Esquemas Pydantic y validaciones compartidas; no tablas ORM.

Flujos críticos

1. **Publicación:** empresa validada crea oferta pendiente; moderación decide; las ofertas activas son elegibles para matching. Los servicios preparan el perfil canónico de oferta.

2. **CV:** carga validada crea CVVersion en almacenamiento privado y actualiza el puntero del postulante; solicitar análisis crea AnalisisCV, TrabajoExtraccion y un EventoSalida. El dispatcher manda el snapshot, la API devuelve un job y el callback firmado entrega el resultado. Se valida identidad, hash y vigencia antes de publicar el perfil.

3. **Edición del perfil:** mantiene versiones e inicia normalización cuando procede. Se conserva la versión de base para no sobreescribir una edición concurrente con un resultado tardío.

4. **Compatibilidad:** se bloquean/recargan perfiles, se comprueba vigencia y configuración, se calcula clave de caché y se guarda snapshot. El outbox entrega match.start; estados y resultado vuelven por WebSocket servidor a servidor; Django avisa a la sesión propietaria. Cancelar registra una cancelación remota y evita completar localmente un trabajo cancelado.

5. **Recomendación:** el resultado completado se cruza con cursos y habilidades locales; no genera otra inferencia.

Decisiones registradas

- ADR-001: persistencia y protección de archivos.

- ADR-002: integración asíncrona e idempotencia.

Estos ADR registran decisiones observadas en código; no atribuyen aprobaciones históricas al equipo.

ADR-001: persistencia relacional y archivos privados

Fecha: 22/10/2026. Estado: decisión observada en la implementación.

Contexto. Usuarios, ofertas y resultados requieren relaciones e integridad. Un CV contiene datos personales y sus bytes no deben quedar accesibles en medios públicos.

Decisión. Usar ORM Django con SQLite para desarrollo y PostgreSQL obligatorio con DEBUG=false. Guardar metadatos/versiones de CV en BD y archivos en private_uploads/cv/; entregar archivos mediante vista autorizada. Usar PROTECT en categorías y configuraciones referenciadas; conservar snapshots y hashes para trazabilidad.

Consecuencias. Se necesita respaldar base y archivos de forma consistente. Una transacción SQL no revierte automáticamente un archivo ya escrito. El borrado de una fila con FileField no implica eliminación física garantizada. El proxy debe bloquear también árboles públicos heredados de CV.

Alternativas. Guardar los bytes en la BD aumenta el tamaño y costo de backup; servir CV bajo /media/ no ofrece el control de acceso requerido. Almacenamiento de objetos privado es una evolución posible, aún no configurada aquí.

Fuentes: ientrabajo/settings/base.py, ia/models.py, usuarios/services.py, usuarios/management/commands/migrar_cv_privados.py.

ADR-002: integración asíncrona con outbox e idempotencia

Fecha: 22/10/2026. Estado: decisión observada en la implementación.

Contexto. Una inferencia puede exceder la duración admisible de una petición, fallar o ser aceptada antes de perder la conexión. Reintentar sin identidad estable puede duplicar trabajo y costo.

Decisión. Guardar el trabajo y EventoSalida en base relacional, enviarlos desde comandos independientes, usar claves de idempotencia y deduplicar callbacks con EventoRecepcion.event_id. Extracción usa HTTP y callback HMAC; matching usa WebSocket autenticado y HTTP para cancelar. El navegador consulta/escucha a Django.

Consecuencias. La entrega admite reintentos y los consumidores deben ser idempotentes; no se promete ejecución exactamente una vez. Se necesitan supervisión de dispatchers, manejo de leases, monitoreo de antigüedad de pendientes y validación de respuestas tardías. El WebSocket de match mantiene una conexión por trabajo mientras lo atiende el dispatcher.

Alternativa. Una llamada LLM síncrona desde la vista alarga la petición y dificulta recuperación. Una cola propia de Django sería otra arquitectura, no la implementada.

Fuentes: ia/workflow.py, ia/management/commands/dispatch_integration_outbox.py, matching/management/commands/dispatch_match_jobs.py, matching/services.py.

Contratos HTTP y WebSocket

Revisión: 22/10/2026. Contratos observados en Django; no se generó ni verificó el OpenAPI de FastAPI. En un entorno con el microservicio, consultar /docs y /openapi.json si su configuración los habilita. No hay un endpoint OpenAPI propio de Django en este repositorio.

Interfaz Django

Las rutas de usuario requieren sesión del postulante. Los POST de navegador requieren CSRF. Una sesión ausente puede redirigir al login (302); los fallos de CSRF/permisos pueden producir HTML 403 y un método no permitido 405. No se debe asumir que todos los errores son JSON.

Método y ruta	Entrada	Respuesta y errores específicos
POST /ia/solicitar-analisis/	Sin cuerpo de negocio; usa el CV actual de la sesión.	202: success, mensaje, analisis_id, version_cv_id; 400 sin CV, 409 conflicto, 502 fallo al preparar archivo, 503 sin servicio configurado.
GET /ia/analisis/{id}/estado/	ID entero, análisis propio.	200: estado, version_cv_id, es_version_actual, mensaje, resultado_url; 404 ajeno/inexistente.
GET /ia/analisis/{id}/	Análisis propio.	HTML de resultado, con controles de vigencia; 404 ajeno/inexistente.
POST /matching/ofertas/{id}/calcular/	ID de oferta activa, sin cuerpo de negocio.	202 pendiente o 200 completado; success, created y estado de match; 409 perfil/configuración no aptos; 404 oferta ausente o no activa.
GET /matching/estado/{uuid}/	UUID propio.	200: estado de match; 404 ajeno/inexistente.
POST /matching/cancelar/{uuid}/	UUID propio.	200: success y estado; 404 ajeno/inexistente.
GET /matching/resultado/{uuid}/	UUID propio.	HTML con detalle, requisitos, desglose y cursos; 404 ajeno/inexistente.
GET /usuarios/cv/{version_id}/descargar/	Versión entera propia.	Archivo privado bajo autorización; no usar URL directa del FileField.

estado_analisis no tiene decorador que limite exclusivamente a GET; se documenta GET como uso del cliente. Las vistas de edición de perfil/habilidades emplean formularios HTML, no constituyen una API JSON pública.

Esquema de estado de compatibilidad

request_id UUID; status en pending, processing, rate_limited, completed, error, cancelled; porcentajes numéricos o null; result objeto solo si completado; error texto o null; scoring_config_hash texto; scoring_config versión y pesos o null; evaluation_date ISO o null; model_version, provider_model_config textos; confidence_available booleano. No se muestra una confianza cuantificada cuando el protocolo no la entrega.

Ejemplo parcial de solicitud encolada (los campos restantes siguen el esquema anterior):

```
{"success": true, "created": true, "request_id": "3e13b5cb-3780-4b4e-aee6-1804f6825827", "status":
"pending", "match_percentage": null, "result": null}
```

Ejemplo desde un navegador con sesión y CSRF disponibles:

```
fetch('/matching/ofertas/42/calcular/', {
  method: 'POST',
  headers: {'X-CSRFToken': csrfToken},
  credentials: 'same-origin'
});
```

Django hacia el servicio de IA

No entregar tokens de servicio al navegador. Las URL son configurables; los siguientes son los paths de la plantilla local.

Transporte / ruta	Cabeceras y contenido	Expectativa del consumidor
POST /api/v1/cv-analyses	Bearer IA_API_TOKEN; Idempotency-Key; multipart cv, analysis_id, candidate_id, user_id, cv_version_id.	HTTP exitoso con JSON que contiene job_id. Django conserva el snapshot privado.
POST /api/v1/offer-analyses	Mismo Bearer/idempotencia; JSON offer_id, source_hash, offer_snapshot.	JSON con job_id.
POST /api/v1/profile-normalizations	Mismo Bearer/idempotencia; JSON candidate_id, source_hash, profile_snapshot.	JSON con job_id.
WS /api/v1/ws/v1/matches	Bearer MATCH_API_TOKEN; evento match.start con payload MatchRequest.	Estados accepted, processing, rate_limited, completed o failed. completed incluye result; failed puede incluir error.
POST /api/v1/matches/{request_id}/cancel	Bearer MATCH_API_TOKEN.	HTTP exitoso; se reintenta si no pudo entregarse.

Extracción usa timeout HTTP de 20 segundos; la espera larga ocurre fuera de la vista web. Matching abre el socket con timeout de conexión de 10 segundos y consulta recepción periódicamente. Los reintentos conservan identidad del trabajo y usan espera creciente con tope según comando.

Callback de extracción

POST /ia/llegue/, exento de CSRF y sin sesión de usuario; autenticado por HMAC. Cabeceras X-IA-Event-Timestamp (segundos Unix) y X-IA-Callback-Signature (hex SHA-256). Firma sobre bytes exactos: timestamp + b'.' + cuerpo, con IA_CALLBACK_HMAC_SECRET. Ventana temporal: diferencia absoluta máxima de 300 segundos.

El cuerpo debe incluir event_id, request_id (clave de idempotencia de TrabajoExtraccion, no asumir que es su UUID), source_hash y el estado/contenido correspondiente. El workflow valida además kind, identificadores de candidato/análisis/oferta y job remoto cuando corresponde. CV: kind candidate_cv; oferta: job_offer; normalización: profile_normalization. Completado exige result compatible con el perfil. No reutilizar solo un ejemplo mínimo como payload válido.

Respuestas: 200 { "success": true, "duplicado": false } o duplicado true; 401 firma/timestamp inválido o vencido; 400 JSON inválido o sin claves básicas; 404 solicitud inexistente/incompatible; 405 método distinto de POST. La validación exhaustiva de payload malformado debe cubrirse con tests de integración.

Eventos hacia el navegador

URL	Autenticación	Mensajes
/ws/matches/{uuid}/	Sesión y propiedad del resultado; origen permitido.	Estado inicial y actualizaciones con el mismo esquema de status_payload. Cierre 4401 anónimo, 4404 ajeno/inexistente.
/ws/cv-analysis/?token=...	Sesión + token Django firmado, salt ia.cv-analysis.websocket, duración máxima 3600 s y usuario coincidente.	Envelope type: cv.analyzed, payload de notificación. Cierre 4401 si no autoriza.
/ws/cv/	Sesión autenticada; grupo privado de usuario.	Payload directo de notificación, sin el envelope anterior.

La notificación CV puede incluir analysis_id, version_cv_id, es_version_actual, status, mensaje y resultado_url. Los dos canales CV tienen formatos diferentes y no deben intercambiarse en el cliente. HTTP de estado permite recuperar el resultado si se interrumpe la conexión.

Versionado y reglas del contenido

ien_contracts/models.py es la fuente verificable de CandidateProfile, JobOffer, MatchRequest y MatchResult. Admite perfiles v1/v2; el flujo de matching construye match.v2, perfiles candidate-profile.v2 y job-offer.v2, pesos para exactamente siete dimensiones y suma 10000 basis points. Los requisitos llevan ID único, categoría y obligatoriedad explícita; los deseables no pueden ser obligatorios. Se conserva fecha de evaluación para credenciales. Los JSON persistidos no crean tablas por cada habilidad o experiencia.

Antes de una integración, comparar esquemas Pydantic con el OpenAPI externo y probar autenticación, idempotencia, callback firmado, errores, cancelación y resultado tardío. No cambiar enums o unidades sin ajustar consumidores y tests.

Modelo de datos de Bolsa de Trabajo IEN

Revisión: 22/10/2026. Fuente: modelos ORM y grafo de migraciones del código local, incluidos cambios sin commit. Se extrajeron **21 entidades propias y 212 campos locales** sin leer filas de personas. Este documento describe el esquema declarado; no acredita que una base productiva tenga todas las migraciones aplicadas.

Convenciones y alcance

PK: clave primaria; FK: clave foránea; 1:1: relación única; N:M: muchos a muchos. null describe la columna; blank describe validación de formularios/modelos. choices y validadores no equivalen por sí solos a CHECK de base. El ORM crea índices para PK, unique y FK según el motor; abajo se distinguen los índices explícitos. Los nombres de columnas FK terminan en _id.

Las tablas de grupos, permisos, sesiones, administración y migraciones son infraestructura Django y no cuentan entre las 21 entidades. Usuario hereda también relaciones N:M a grupos/permisos. Moderación, emails e ien_contracts no agregan entidades propias vigentes. La BD almacena rutas de archivos, no sus bytes.

Lectura del dominio

- Usuario puede tener cero o un Postulante y cero o un Oferente; no hay una restricción que haga estos perfiles excluyentes.

- Oferta.empresa referencia Usuario. La autorización de empresa aprobada vive en la aplicación, no en una FK a Oferente.

- Categoria se protege mientras la usen Oferta, Curso o Habilidad. Las habilidades de una oferta son texto/JSON; no existe una FK desde cada requisito al catálogo Habilidad.

- Curso declara habilidades mediante tabla puente automática cursos_curso_habilidades, con pareja única curso/habilidad.

- Postulante tiene muchas CVVersion; el puntero opcional cv_version_actual marca la vigente. AnalisisCV y PerfilCV pueden conservar referencias a una versión.

- PerfilCV y PerfilOferta tienen una relación 1:1 con su entidad y muchas versiones históricas.

- ResultadoCompatibilidad enlaza candidato/oferta y, opcionalmente, configuración. Sus versiones/hashes de perfil son valores y snapshots, no FK a VersionPerfilCV/VersionPerfilOferta.

- EventoSalida enlaza trabajos por JSON (job_id/match_id/request_id); EventoRecepcion deduplica por event_id. No dibujar una FK inexistente.

- No hay entidades de candidatura, entrevista, matrícula o contratación. Las favoritas no registran un proceso de selección.

Diagramas ER

El diagrama completo editable contiene todas las FK entre entidades propias y la tabla puente de cursos. Los diagramas parciales siguientes facilitan la lectura. || es uno, o| cero o uno y o{ cero o muchos. Los enlaces de pertenencia no garantizan por sí mismos la coherencia entre dos caminos relacionados.

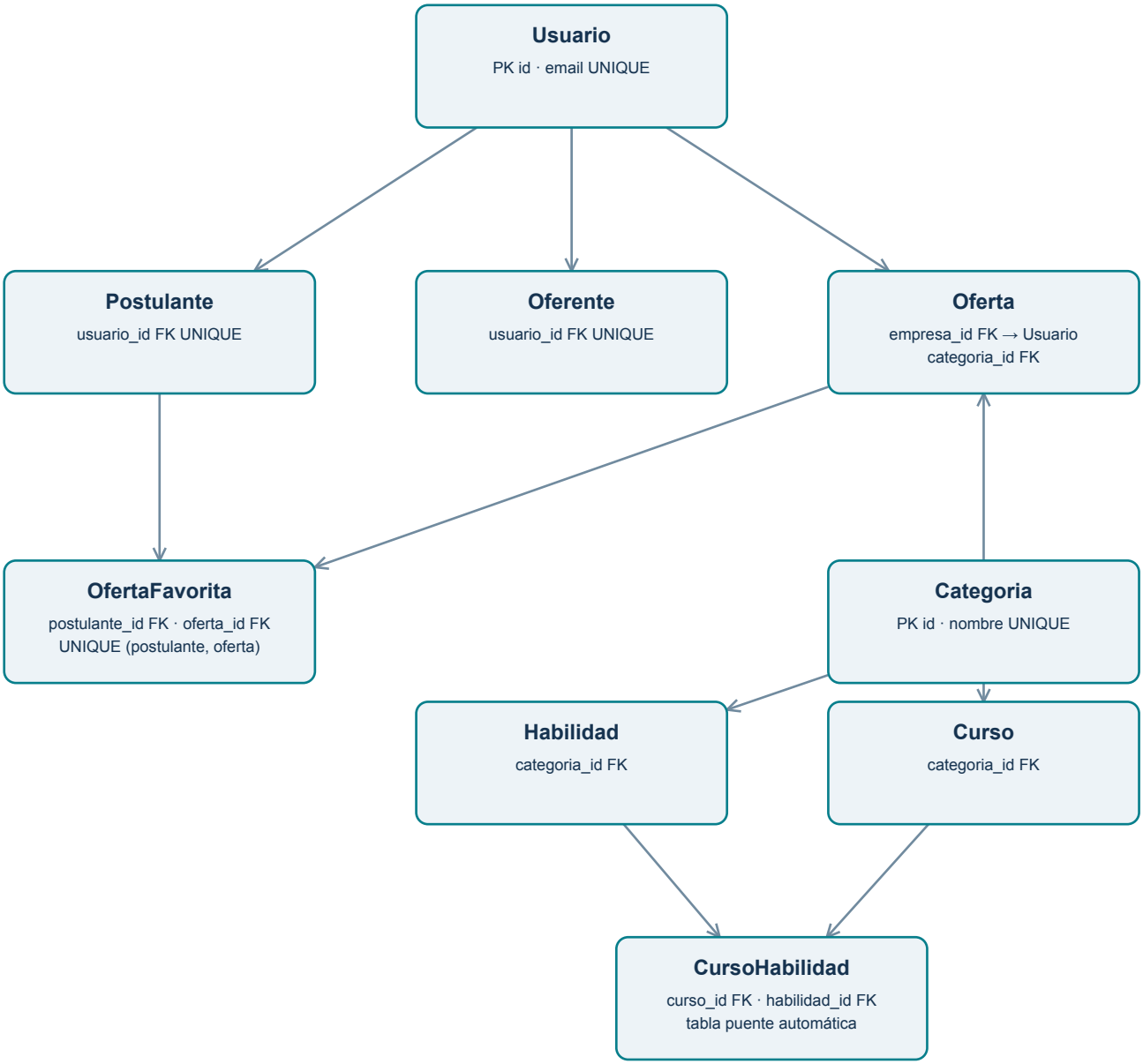
Identidad, ofertas y formación

CV, perfiles e integración

Compatibilidad y ponderaciones

Identidad, ofertas y formación

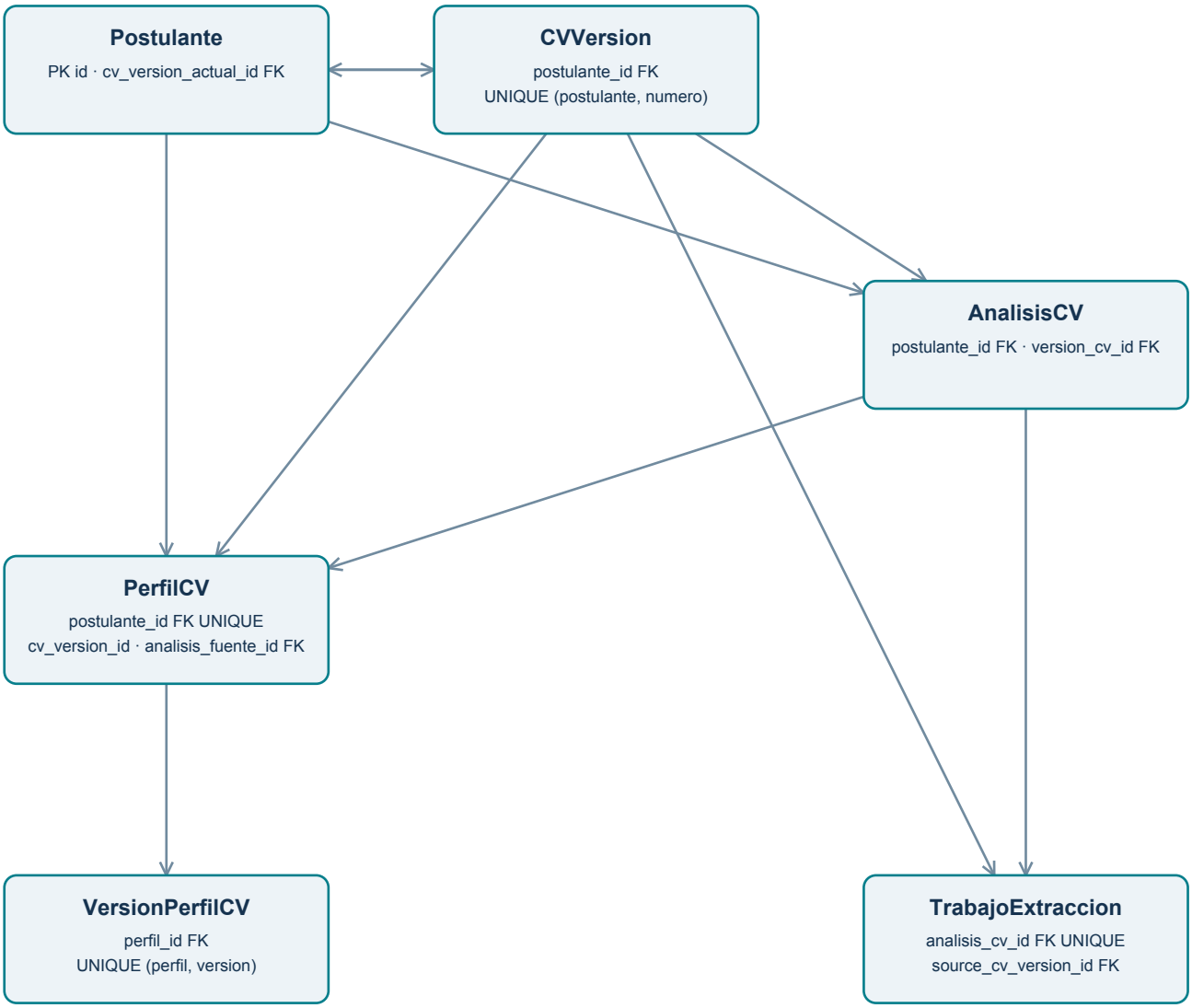
Modelo relacional declarado · 22/10/2026



Flechas: entidad referenciada → entidad que contiene la FK. Los perfiles son opcionales 1:1. Las demás relaciones admiten múltiples filas hijas; la tabla puente resuelve Curso N:M Habilidad. El ER editable conserva las cardinalidades exactas.

CV y perfil del candidato

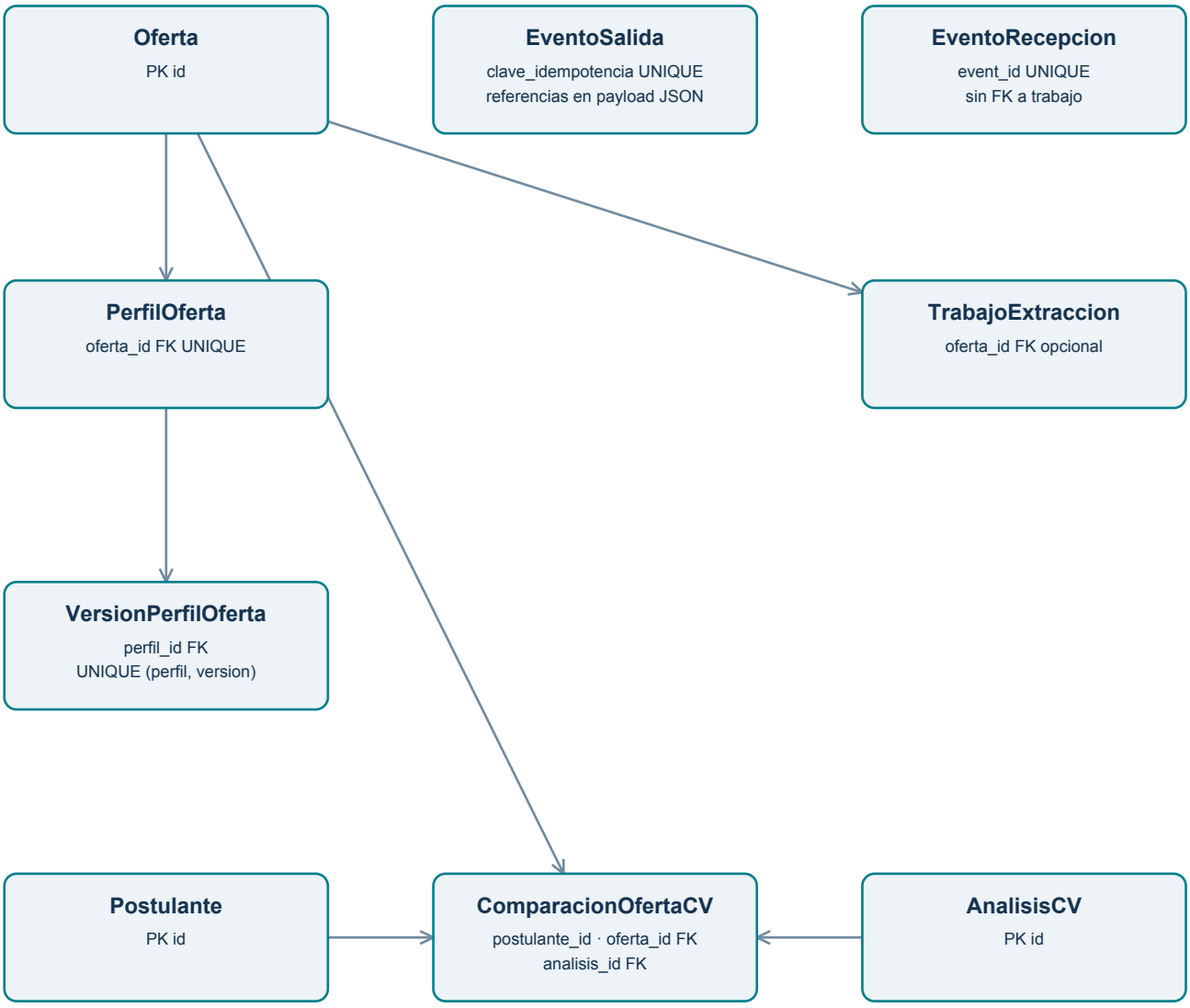
Modelo relacional declarado · 22/10/2026



El puntero CV actual y las FK a versiones admiten NULL donde lo indica el diccionario. El ciclo Postulante ↔ CVVersion es intencional. La pertenencia del CV actual al mismo postulante la controlan los servicios.

Ofertas e integración

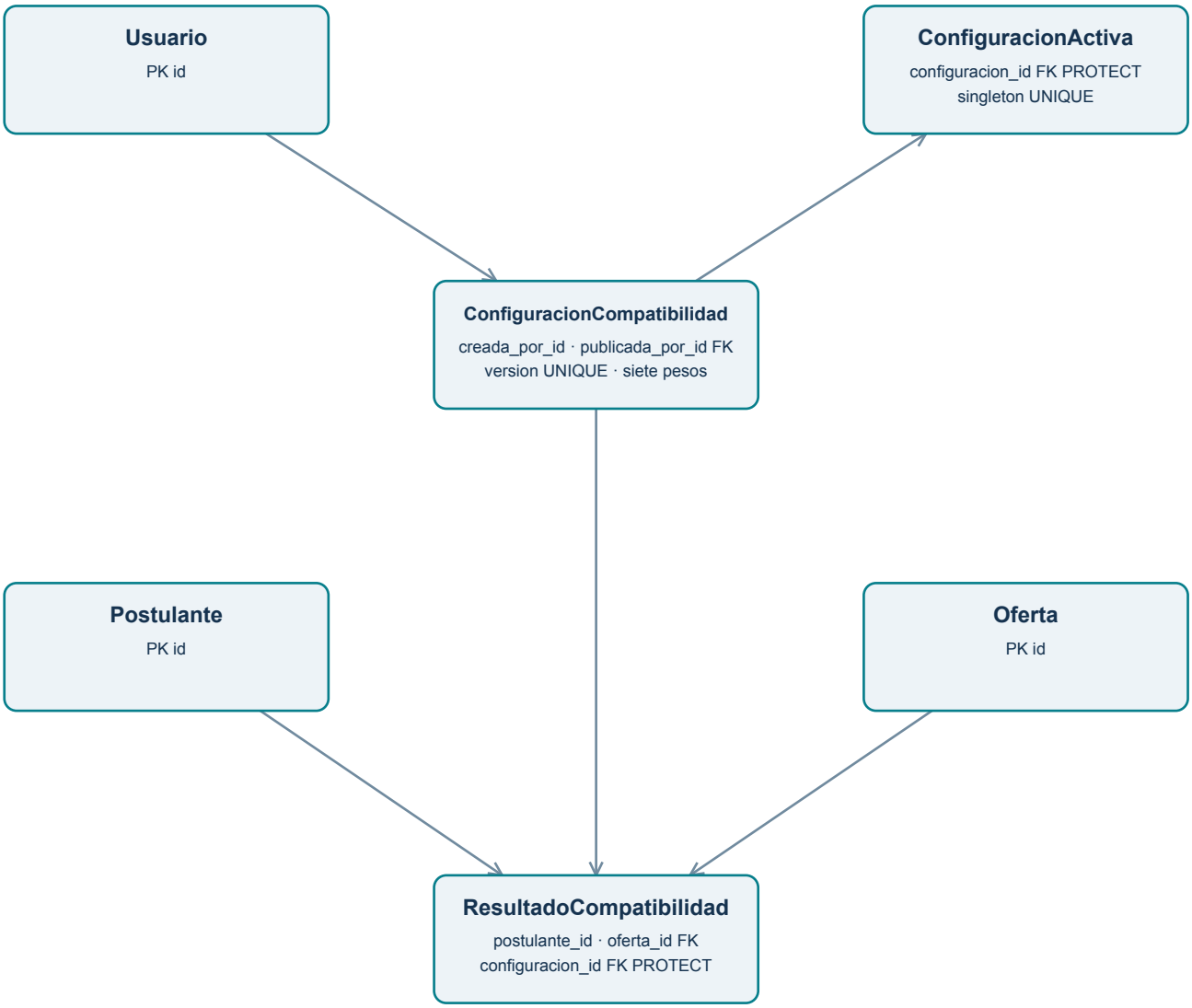
Modelo relacional declarado · 22/10/2026



Outbox e inbox no tienen FK declaradas al trabajo: sus vínculos son lógicos. ComparacionOfertaCV conserva la estructura histórica; el flujo actual calcula mediante ResultadoCompatibilidad. Las relaciones de TrabajoExtraccion con CV están en la lámina anterior.

Compatibilidad y configuración

Modelo relacional declarado · 22/10/2026



Resultado conserva snapshots, versiones numéricas y hashes. No hay FK a versiones de perfil. request_id y cache_key son únicos. Índice compuesto: postulante, oferta, estado. ConfiguracionActiva es singleton por servicio/convención, no por límite SQL de una fila.

Diccionario de entidades y campos

Tipos ORM portables: la representación SQL depende de SQLite/PostgreSQL. JSONField contiene documentos validados por servicios/Pydantic, no relaciones normalizadas. Los defaults de fecha, UUID o versión se evalúan en aplicación cuando son funciones.

Usuario

Tabla: usuarios_usuario. Identidad y permisos. Email único como login; hereda campos de autenticación Django y no usa username.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
password	CharField(128)	no / no	-
last_login	DateTimeField	sí / sí	-
is_superuser	BooleanField	no / no	default=False
first_name	CharField(150)	no / sí	-
last_name	CharField(150)	no / sí	-
is_staff	BooleanField	no / no	default=False
is_active	BooleanField	no / no	default=True
date_joined	DateTimeField	no / no	default=now
email	CharField(254)	no / no	UNIQUE

N:M groups → Group, tabla usuarios_usuario_groups; columnas: id, usuario_id, group_id.

N:M user_permissions → Permission, tabla usuarios_usuario_user_permissions; columnas: id, usuario_id, permission_id.

Postulante

Tabla: usuarios_postulante. Datos personales y puntero opcional a la versión actual de CV.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
usuario / usuario_id	OneToOneField	no / no	UNIQUE; 1:1 → Usuario; CASCADE
dni	CharField(8)	sí / sí	-
fecha_nacimiento	DateField	sí / sí	-
telefono	CharField(20)	sí / sí	-
direccion	CharField(200)	sí / sí	-
cv	FileField(100)	sí / sí	-
cv_fecha_carga	DateTimeField	sí / sí	-
cv_version_actual / cv_version_actual_id	ForeignKey	sí / sí	FK → CVVersion; SET_NULL
foto	FileField(100)	sí / sí	-

Oferente

Tabla: usuarios_oferente. Perfil empresarial y estado de validación; CUIT validado por aplicación, sin unicidad de BD.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
usuario / usuario_id	OneToOneField	no / no	UNIQUE; 1:1 → Usuario; CASCADE
estado_validacion	CharField(20)	no / no	default=pendiente; opciones: pendiente, aprobado, rechazado
motivo_rechazo	TextField	sí / sí	-
nombre_empresa	CharField(200)	no / no	-
cuit	CharField(13)	no / no	-
logo	FileField(100)	sí / sí	-
banner	FileField(100)	sí / sí	-
descripcion	TextField	sí / sí	-
industria	CharField(100)	sí / sí	-
tamano_empresa	CharField(20)	sí / sí	opciones: 1-10, 11-50, 51-200, 201-1000, 1000+
ubicacion	CharField(200)	sí / sí	-
anio_fundacion	IntegerField	sí / sí	-
sitio_web	CharField(300)	sí / sí	-
telefono	CharField(50)	sí / sí	-

Curso

Tabla: cursos_curso. Formación ofrecida por el instituto; relación N:M con habilidades.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
nombre	CharField(120)	no / no	-
categoria / categoria_id	ForeignKey	no / no	FK → Categoria; PROTECT
imagen	FileField(100)	no / no	-
descripcion	TextField	no / no	-
duracion	CharField(100)	no / no	-
horario	CharField(200)	no / no	-
tipo	CharField(20)	no / no	default=Presencial; opciones: Presencial, Virtual, Hibrido
url_externa	CharField(200)	sí / sí	-
activo	BooleanField	no / no	default=True

N:M habilidades → Habilidad, tabla cursos_curso_habilidades; columnas: id, curso_id, habilidad_id.

Oferta

Tabla: ofertas_oferta. Publicación laboral propiedad de Usuario, con requisitos libres/estructurados y moderación.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
empresa / empresa_id	ForeignKey	no / no	FK → Usuario; CASCADE
categoria / categoria_id	ForeignKey	no / no	FK → Categoria; PROTECT
titulo	CharField(100)	no / no	-
nombre_puesto	CharField(150)	no / no	-
ubicacion	CharField(100)	no / no	-
modalidad	CharField(20)	no / no	default=presencial; opciones: presencial, remoto, hibrido
descripcion	TextField	no / no	-
habilidades_duras	TextField	no / sí	default=
habilidades_blandas	TextField	no / sí	default=
habilidades_requeridas	TextField	no / no	-
requisitos_obligatorios	TextField	no / sí	default=
requisitos_deseables	TextField	no / sí	default=
requisitos_estructurados	JSONField	no / sí	default=list
requiere_revision_requisitos	BooleanField	no / no	default=True
experiencia_clasificacion	CharField(16)	no / no	default=no_solicitado; opciones: obligatorio, deseable, no_solicitado
educacion_clasificacion	CharField(16)	no / no	default=no_solicitado; opciones: obligatorio, deseable, no_solicitado
experiencia_requerida	CharField(20)	no / no	default=sin-experiencia; opciones: sin-experiencia, menos-1, 1-2, 2-3, 3-5, 5+
nivel_educativo	CharField(30)	no / no	opciones: secundario, terciario, universitario, posgrado
es_confidencial	BooleanField	no / no	default=False
estado	CharField(20)	no / no	default=pendiente; opciones: pendiente, activa, rechazada, finalizada
fecha_publicacion	DateTimeField	no / no	default=now
fecha_cierre	DateTimeField	no / no	-
motivo_rechazo	TextField	sí / sí	-

OfertaFavorita

Tabla: ofertas_ofertafavorita. Asociación única de un postulante con una oferta guardada; no es una postulación.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
postulante / postulante_id	ForeignKey	no / no	FK → Postulante; CASCADE
oferta / oferta_id	ForeignKey	no / no	FK → Oferta; CASCADE
creado_en	DateTimeField	no / sí	-

Restricción explícita: <UniqueConstraint: fields=('postulante', 'oferta') name='unico_postulante_oferta_favorita'>.

Categoria

Tabla: categorias_categoria. Clasificación compartida de ofertas, cursos y habilidades.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
nombre	CharField(100)	no / no	UNIQUE

Habilidad

Tabla: categorias_habilidad. Catálogo de competencias duras/blandas; una categoría por habilidad.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
categoria / categoria_id	ForeignKey	no / no	FK → Categoria; PROTECT
nombre	CharField(100)	no / no	-
tipo_habilidad	CharField(1)	no / no	default=D; opciones: B, D

Unicidad conjunta: [['nombre', 'categoria']].

CVVersion

Tabla: ia_cvversion. Archivo privado y metadatos de una carga aceptada. La inmutabilidad es una intención de servicio; no una restricción SQL contra UPDATE.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
postulante / postulante_id	ForeignKey	no / no	FK → Postulante; CASCADE
numero	PositiveIntegerField	no / no	-
archivo	FileField(100)	no / no	-
nombre_original	CharField(255)	no / no	-
sha256	CharField(64)	no / no	-
formato	CharField(12)	no / no	-
tamano	PositiveBigIntegerField	no / no	-
creado_en	DateTimeField	no / sí	-

Restricción explícita: <UniqueConstraint: fields=('postulante', 'numero') name='ia_cv_version_numero_unico'>.

AnalisisCV

Tabla: ia_analisiscv. Solicitud histórica de análisis de una versión de CV y respuesta del servicio.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
postulante / postulante_id	ForeignKey	no / no	FK → Postulante; CASCADE
version_cv / version_cv_id	ForeignKey	sí / sí	FK → CVVersion; SET_NULL
perfil_snapshot	JSONField	sí / sí	-
estado	CharField(12)	no / no	default=pendiente; opciones: pendiente, completado, error
respuesta	JSONField	sí / sí	-
error	TextField	no / sí	-
creado_en	DateTimeField	no / sí	-
respondido_en	DateTimeField	sí / sí	-

PerfilCV

Tabla: ia_perfilcv. Perfil canónico editable de un postulante, con versión y estado de normalización.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
postulante / postulante_id	OneToOneField	no / no	UNIQUE; 1:1 → Postulante; CASCADE
cv_version / cv_version_id	ForeignKey	sí / sí	FK → CVVersion; SET_NULL
contenido	JSONField	sí / sí	-
version	PositiveIntegerField	no / no	default=0
source_hash	CharField(64)	no / sí	-
schema_version	CharField(64)	no / no	default=candidate-profile.v1
estado	CharField(20)	no / no	default=pending; opciones: pending, processing, ready, error, proposal_ready
origen	CharField(24)	no / no	default=extraction
analisis_fuente / analisis_fuente_id	ForeignKey	sí / sí	FK → AnalisisCV; SET_NULL
actualizado_en	DateTimeField	no / sí	-

VersionPerfilCV

Tabla: ia_versionperfilcv. Snapshot histórico JSON de una versión del perfil canónico de candidato.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
perfil / perfil_id	ForeignKey	no / no	FK → PerfilCV; CASCADE
version	PositiveIntegerField	no / no	-
contenido	JSONField	no / no	-
content_hash	CharField(64)	no / no	-
origen	CharField(24)	no / no	-
creado_en	DateTimeField	no / sí	-

Restricción explícita: <UniqueConstraint: fields=('perfil', 'version') name='ia_perfil_cv_version_unica'>.

PerfilOferta

Tabla: ia_perfiloferta. Perfil canónico de una oferta para alimentar el comparador.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
oferta / oferta_id	OneToOneField	no / no	UNIQUE; 1:1 → Oferta; CASCADE
contenido	JSONField	sí / sí	-
version	PositiveIntegerField	no / no	default=0
source_hash	CharField(64)	no / sí	-
schema_version	CharField(64)	no / no	default=job-offer.v1
estado	CharField(20)	no / no	default=pending; opciones: pending, processing, ready, error, proposal_ready
actualizado_en	DateTimeField	no / sí	-

VersionPerfilOferta

Tabla: ia_versionperfiloferta. Snapshot histórico JSON de una versión del perfil de oferta.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
perfil / perfil_id	ForeignKey	no / no	FK → PerfilOferta; CASCADE
version	PositiveIntegerField	no / no	-
contenido	JSONField	no / no	-
content_hash	CharField(64)	no / no	-
creado_en	DateTimeField	no / sí	-

Restricción explícita: <UniqueConstraint: fields=('perfil', 'version') name='ia_perfil_oferta_version_unica'>.

TrabajoExtraccion

Tabla: ia_trabajoextraccion. Seguimiento de extracción de CV/oferta o normalización; conserva snapshot y claves de entrega.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
request_id	UUIDField(32)	no / no	UNIQUE; default=uuid4
clave_idempotencia	CharField(180)	no / no	UNIQUE
tipo	CharField(32)	no / no	opciones: candidate_cv, job_offer, normalize_profile
analisis_cv / analisis_cv_id	OneToOneField	sí / sí	UNIQUE; 1:1 → AnalisisCV; CASCADE
oferta / oferta_id	ForeignKey	sí / sí	FK → Oferta; CASCADE
source_hash	CharField(64)	no / no	-
prompt_version	CharField(64)	no / no	default=extraction.v1
remote_job_id	CharField(64)	no / sí	-
cv_snapshot	FileField(100)	sí / sí	-
profile_snapshot	JSONField	sí / sí	-
profile_version_base	PositiveIntegerField	sí / sí	-
source_cv_version / source_cv_version_id	ForeignKey	sí / sí	FK → CVVersion; SET_NULL
estado	CharField(24)	no / no	default=pending; opciones: pending, sending, accepted, processing, completed, error, delivery_unknown
intentos	PositiveIntegerField	no / no	default=0
error	TextField	no / sí	-
creado_en	DateTimeField	no / sí	-
actualizado_en	DateTimeField	no / sí	-

EventoSalida

Tabla: ia_eventosalida. Outbox persistente: intención de envío/reintento. Las referencias del payload son lógicas y no claves foráneas.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
tipo	CharField(24)	no / no	opciones: extract_cv, extract_offer, normalize_profile, match, cancel_match
clave_idempotencia	CharField(180)	no / no	UNIQUE
payload	JSONField	no / no	default=dict
estado	CharField(16)	no / no	default=pending; opciones: pending, sending, sent, error
intentos	PositiveIntegerField	no / no	default=0

Campo / columna	Tipo	Null / blank	Reglas declaradas
proximo_intento_en	DateTimeField	sí / sí	-
error	TextField	no / sí	-
creado_en	DateTimeField	no / sí	-
actualizado_en	DateTimeField	no / sí	-

EventoRecepcion

Tabla: ia_eventorecepcion. Inbox de deduplicación: conserva identificador de evento y fecha, sin FK al trabajo.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
event_id	CharField(100)	no / no	UNIQUE
recibido_en	DateTimeField	no / sí	-

ComparacionOfertaCV

Tabla: ia_comparacionofertacv. Persistencia de comparación histórica por postulante/oferta/análisis. La ruta histórica actual delega en matching; no confundir esta tabla con ResultadoCompatibilidad.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
postulante / postulante_id	ForeignKey	no / no	FK → Postulante; CASCADE
oferta / oferta_id	ForeignKey	no / no	FK → Oferta; CASCADE
analisis / analisis_id	ForeignKey	no / no	FK → AnalisisCV; CASCADE
porcentaje	PositiveSmallIntegerField	no / no	default=0
resultado	JSONField	no / no	default=dict
creado_en	DateTimeField	no / sí	-
actualizado_en	DateTimeField	no / sí	-

Restricción explícita: <UniqueConstraint: fields=('postulante', 'oferta', 'analisis') name='ia_comparacion_unica_por_oferta_y_analisis'>.

ConfiguracionCompatibilidad

Tabla: matching_configuracioncompatibilidad. Rúbrica con siete pesos. Publicada es inmutable por métodos del modelo; borradores siguen editables.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
version	CharField(64)	no / no	UNIQUE; default=nueva_version_configuracion
technical_skills	DecimalField(5,2)	no / no	default=35.00

Campo / columna	Tipo	Null / blank	Reglas declaradas
soft_skills	DecimalField(5,2)	no / no	default=15.00
experience	DecimalField(5,2)	no / no	default=20.00
education	DecimalField(5,2)	no / no	default=10.00
languages	DecimalField(5,2)	no / no	default=5.00
logistics	DecimalField(5,2)	no / no	default=5.00
desirables	DecimalField(5,2)	no / no	default=10.00
es_borrador	BooleanField	no / no	default=True
creada_por / creada_por_id	ForeignKey	sí / sí	FK → Usuario; SET_NULL
publicada_por / publicada_por_id	ForeignKey	sí / sí	FK → Usuario; SET_NULL
creada_en	DateTimeField	no / sí	-
publicada_en	DateTimeField	sí / sí	-

ConfiguracionActiva

Tabla: matching_configuracionactiva. Puntero a la rúbrica activa. singleton es único por valor; la BD no fuerza una única fila total.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
configuracion / configuracion_id	ForeignKey	no / no	FK → ConfiguracionCompatibilidad; PROTECT
singleton	CharField(16)	no / no	UNIQUE; default=actual

ResultadoCompatibilidad

Tabla: matching_resultadocompatibilidad. Resultado o trabajo de matching con identidad, caché, snapshots, versiones, pesos y métricas.

Campo / columna	Tipo	Null / blank	Reglas declaradas
id	AutoField	no / sí	PK
postulante / postulante_id	ForeignKey	no / no	FK → Postulante; CASCADE
oferta / oferta_id	ForeignKey	no / no	FK → Oferta; CASCADE
request_id	UUIDField(32)	no / no	UNIQUE; default=uuid4
cache_key	CharField(64)	no / no	UNIQUE
estado	CharField(20)	no / no	default=pending; opciones: pending, processing, rate_limited, completed, error, cancelled
perfil_cv_version	PositiveIntegerField	no / no	-
perfil_oferta_version	PositiveIntegerField	no / no	-
perfil_cv_hash	CharField(64)	no / no	-

Campo / columna	Tipo	Null / blank	Reglas declaradas
perfil_oferta_hash	CharField(64)	no / no	-
rubric_version	CharField(64)	no / no	default=match-rubric.v2
matcher_prompt_version	CharField(64)	no / no	default=match-groq.v2
model_version	CharField(120)	no / no	default=provider-managed
provider_model_config	CharField(120)	no / no	default=
configuracion / configuracion_id	ForeignKey	sí / sí	FK → ConfiguracionCompatibilidad; PROTECT
scoring_config_hash	CharField(64)	no / no	default=
scoring_weights	JSONField	no / no	default=dict
evaluation_date	DateField	sí / sí	-
resultado	JSONField	sí / sí	-
candidate_snapshot	JSONField	no / no	default=dict
offer_snapshot	JSONField	no / no	default=dict
match_percentage	DecimalField(5,2)	sí / sí	-
coverage_percentage	DecimalField(5,2)	sí / sí	-
confidence_percentage	DecimalField(5,2)	sí / sí	-
error	TextField	no / sí	-
creado_en	DateTimeField	no / sí	-
actualizado_en	DateTimeField	no / sí	-

Índice explícito matching_re_postula_c69c51_idx: postulante, oferta, estado.

Integridad y límites de las reglas

Regla	Mecanismo efectivo / límite
Un email por Usuario	Columna unique. No implica normalización completa ni unicidad ignorando mayúsculas en todos los motores.
Un número de CV por postulante	UniqueConstraint (postulante, numero). El hash no es único global.
Una versión por perfil	UniqueConstraint (perfil, version) en ambos históricos.
Una favorita por postulante/oferta	UniqueConstraint explícita.
Una comparación histórica por terna	UniqueConstraint (postulante, oferta, analisis).
No duplicar envíos/eventos	TrabajoExtraccion.clave_idempotencia, EventoSalida.clave_idempotencia y EventoRecepcion.event_id únicos.
No duplicar caché/identidad de match	ResultadoCompatibilidad.cache_key y request_id únicos; puede haber varios resultados por candidato/oferta con otra clave.
Una habilidad por nombre y categoría	unique_together; no incluye tipo_habilidad.
Proteger catálogos y rúbricas en uso	PROTECT en FK a categorías y configuraciones referenciadas.
Suma de pesos y publicación inmutable	clean/save/delete y servicio de publicación; no CHECK SQL de suma ni protección frente a SQL directo/QuerySet.update.
CV actual pertenece al postulante	Coherencia de servicios; la FK sola no verifica propietario. Igual precaución para comparaciones con análisis ajeno.
Un único perfil de cada tipo por usuario	OneToOne; no impide tener ambos tipos de perfil.
Un único análisis pendiente	AnalisisCV tiene constraints vacías; no atribuir la antigua restricción de migración 0002 al esquema actual. La coordinación reside en workflow/bloqueos.

CASCADE elimina filas dependientes al borrar la entidad referenciada; SET_NULL conserva el registro sin referencia; PROTECT impide el borrado mientras existan dependencias. Aplicar estos efectos antes de definir una política institucional de supresión.

Índices críticos

El índice compuesto explícito de ResultadoCompatibilidad cubre (postulante, oferta, estado). Los índices por unicidad sustentan búsquedas de email, clave de caché, request_id, idempotencia y event_id. Los dispatchers consultan outbox por tipo/estado/fecha: no hay un índice compuesto explícito para ese patrón en EventoSalida. Evaluar EXPLAIN y volumen antes de proponerlo; esta entrega no modifica el esquema ni promete rendimiento medido.

Estados persistidos

Los valores exactos de enums están en el diccionario. No unificarlos al integrar: AnalisisCV usa pendiente/completado/error, mientras trabajos y matching usan valores en inglés. TrabajoExtraccion incluye delivery_unknown; PerfilCV/PerfilOferta incluyen proposal_ready; ResultadoCompatibilidad incluye cancelled y rate_limited. EventoSalida distingue pending/sending/sent/error. Una elección de TextChoices no sustituye la validación de transiciones de los servicios.

Migraciones y evolución

Tomar los archivos de migración como historial autoritativo y los modelos como intención actual. Las ramas históricas de usuarios e IA ya tienen migraciones de merge; no renumerarlas. Las hojas locales detectadas son:

Aplicación	Migración terminal local
categorias	0006_proteger_categorias_en_uso
cursos	0007_curso_habilidades
ia	0009_alter_eventosalida_tipo
matching	0007_alter_resultadocompatibilidad_estado
ofertas	0015_oferta_educacion_clasificacion_and_more
usuarios	0016_postulante_cv_version_actual_postulante_foto

IA 0009 y matching 0007 existían como archivos locales sin seguimiento al iniciar esta revisión. Deben incluirse junto con sus cambios de código al preparar la entrega correspondiente; no se crearon ni aplicaron migraciones aquí.

```
.venv/bin/python manage.py makemigrations --check --dry-run
.venv/bin/python manage.py showmigrations
.venv/bin/python manage.py migrate --plan
```

Los dos últimos comandos consultan el entorno configurado: no confundir ese resultado con otro ambiente. Para una evolución: cambiar modelo, generar/revisar migración y reversibilidad, probar sobre base temporal, respaldar y aplicar en una ventana definida. Documentar cualquier transformación de datos y compatibilidad de versiones.

Datos personales, archivos y acceso

Email, nombres, DNI, domicilio, teléfono, nacimiento, CV, foto, experiencias y evidencias del perfil son datos personales. CUIT y contactos de empresa también requieren acceso acorde a finalidad. Los JSON de análisis/resultados pueden replicar datos del CV; no proteger solo el archivo original. Hashes y nombres de archivo no anonimizan el registro.

La descarga CV exige autorización de propietario; PrivateCVStorage.url() rechaza una URL pública. Almacenamiento activo: private_uploads/cv/ y snapshots/ bajo ese árbol. El campo Postulante.cv conserva compatibilidad heredada y dispone de un comando de migración privado. El proxy debe bloquear también los directorios públicos anteriores. Usuario conserva contraseñas mediante hashing Django, no cifrado reversible; secretos de integración pertenecen al entorno, no a estas tablas.

Acceso de resultado/estado de IA y matching filtrado por postulante; moderación restringida a staff. Las FK no implementan seguridad por filas para consultas SQL directas. Aplicar cuenta de BD con privilegios mínimos, backup restringido y registros sin contenido de CV ni tokens.

Retención y eliminación

No se identificó una política temporal ni tarea de purga automática de CVVersion, históricos, outbox/inbox o resultados. No se afirma cumplimiento de un plazo institucional. Deben acordarse propósito, duración, bloqueo por solicitudes, borrado de copias en JSON/archivos/backup y responsable.

El método de eliminación del CV actual no debe confundirse con purga de toda la historia. La eliminación física de archivos no está garantizada por un CASCADE de BD; evaluar referencias activas y snapshots antes de limpiar. No eliminar EventoRecepcion antes del horizonte de reintento del productor: podría reaplicarse un evento viejo. Definir la retención de ambos lados conjuntamente.

Verificación y mantenimiento

El inventario se obtuvo por introspección de _meta y MigrationLoader, sin consultas a filas de negocio. Cada cambio en modelos/migraciones exige actualizar diccionario, cardinalidades, restricciones, índices y acceso/retención. Para revisar un entorno concreto, comparar showmigrations y el esquema de esa base con el modelo declarado.

Despliegue y operación

Revisión: 22/10/2026. El repositorio tiene configuración y comandos locales, pero no una plataforma productiva ni unidades de servicio declaradas. Este runbook es una base operativa; deploy, rollback y restauración productivos requieren ensayo en staging y registro de evidencia.

Ambientes y configuración

Desarrollo: SQLite permitido, DEBUG=true, Django 8000, FastAPI externo 8001 y Redis 6379 según plantilla. Staging/producción: PostgreSQL, DEBUG=false, dominio permitido, secretos propios, TLS y proxy con soporte WebSocket. No copiar datos personales a pruebas.

Partir de .env.example. Grupos obligatorios: SECRET_KEY, ALLOWED_HOSTS, DB_ENGINE=postgresql, DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT, DB_SSLMODE; integración IA_API_URL, IA_OFFER_API_URL, IA_API_TOKEN, IA_CALLBACK_HMAC_SECRET, MATCH_API_URL, MATCH_API_HTTP_URL, MATCH_API_TOKEN; Redis CHANNEL_REDIS_URL, MATCH_CACHE_REDIS_URL; seguridad CSRF_TRUSTED_ORIGINS, TRUST_PROXY_HEADERS, SECURE_SSL_REDIRECT, SECURE_HSTS_SECONDS; correo opcional EMAIL_HOST_USER/EMAIL_HOST_PASSWORD y contacto IEN_CONTACT_EMAIL.

Los tokens salientes deben coincidir con la configuración de servicio de FastAPI y el secreto HMAC con el de callbacks. IA_CALLBACK_URL está reservada y no se usa en el flujo actual. MATCH_CACHE_TTL_SECONDS predetermina 604800 s y MATCH_REQUESTS_PER_HOUR 20; son parámetros técnicos, no política de retención de CV. No se requiere configurar un proveedor LLM en Django: corresponde al servicio externo.

Arranque local de Django y despachadores

Seguir primero el README para entornos, migraciones, Redis, FastAPI y Celery. Además de la web, ejecutar estos dos procesos en terminales independientes:

```
.venv/bin/python manage.py run_integration_dispatcher --kind extraction --interval 2
.venv/bin/python manage.py run_integration_dispatcher --kind match --interval 2
```

Para una entrega acotada, sin bucle continuo:

```
.venv/bin/python manage.py dispatch_integration_outbox --once
.venv/bin/python manage.py dispatch_match_jobs --once
```

Estos comandos entregan trabajos reales del entorno configurado; no usarlos como comprobación de solo lectura en producción. Redis y workers Celery del servicio externo se inician en su propio repositorio. El servidor debe ser ASGI para notificaciones; channels[daphne] está declarado y también se puede iniciar explícitamente:

```
.venv/bin/daphne -b 127.0.0.1 -p 8000 ientrabajo.asgi:application
```

Servicios productivos propuestos

Supervisar por separado web ASGI, dispatcher de extracción y dispatcher de match, además de PostgreSQL, Redis y el stack externo de IA. Definir usuario de servicio sin privilegios, directorio de trabajo, entorno privado, reinicio con backoff y logs por proceso. No se incluyeron unidades systemd porque aún no están definidos host, usuario ni rutas de instalación.

El proxy sirve estáticos/medios públicos, reenvía HTTP y Upgrade de WebSocket y bloquea /media/postulantes/cv/ y /media/ia/extraction_snapshots/. Nunca publicar private_uploads/. Validar orígenes/CSRF y cabeceras de proxy; confiar en X-Forwarded-Proto solo desde un proxy controlado.

Secuencia de despliegue propuesta

1. Identificar revisión de Django y versión compatible de FastAPI; ejecutar pruebas y revisión del plan de migraciones en staging.

2. Respaldar PostgreSQL, private_uploads/ y media/ en una ventana consistente; conservar configuración cifrada y referencias de release.

3. Detener entregas/escrituras para cambios incompatibles; instalar dependencias en un entorno de release. No ejecutar dos versiones incompatibles de dispatchers sobre la misma outbox.

4. Revisar y aplicar:

```
.venv/bin/python manage.py check --deploy
.venv/bin/python manage.py migrate --plan
.venv/bin/python manage.py migrate
.venv/bin/python manage.py collectstatic --noinput
```

5. Arrancar/reiniciar servicios supervisados y comprobar login, oferta, descarga privada, extracción de archivo sintético, callback y match. Verificar WebSocket y consulta HTTP de estado.

6. Registrar release, migraciones, resultados, pendientes y responsable. Habilitar tráfico completo después del smoke.

Rollback propuesto

Detener web/dispatchers afectados y revisar si el código anterior es compatible con el esquema. Si lo es, restaurar la release previa y reiniciar. Si no lo es, usar únicamente una migración inversa ensayada o restaurar backup consistente de BD y archivos. Una restauración pierde escrituras posteriores al backup: acordar la ventana antes de realizarla. No ejecutar migraciones inversas ni borrar eventos automáticamente. Mantener colas/callbacks de versiones incompatibles aislados durante la recuperación.

Backup y restauración

Ejemplo para PostgreSQL con credenciales mediante .pgpass o mecanismo de secretos, sin contraseña en comandos:

```
pg_dump --format=custom --file=bolsa-backup.dump --host=HOST --username=USUARIO BASE
pg_restore --dbname=BASE_RESTAURACION --host=HOST --username=USUARIO --no-owner bolsa-backup.dump
```

Sustituir HOST/USUARIO/BASE; BASE_RESTAURACION debe ser una base vacía y aislada. Respaldar también archivos de CV, snapshots e imágenes con acceso restringido. En SQLite local, detener escritores antes de copiar; no tratar una copia en caliente del archivo como respaldo consistente. Redis no reemplaza el backup relacional; la caché puede reconstruirse, pero la cola externa necesita su estrategia propia.

Ensayar restauración en ambiente aislado: verificar migraciones, conteos, integridad de relaciones, hashes/lectura de archivos y permisos de descarga. Medir y acordar RPO/RTO, frecuencia, cifrado, retención y responsable. Aún no hay una restauración productiva acreditada en esta entrega.

Logs, métricas y recuperación

No hay un sistema central de observabilidad declarado. Capturar stdout/stderr de procesos y logs Django; incluir request_id/event_id y estado, evitando CV, JSON personales y tokens.

Síntoma	Revisión / recuperación
Trabajo pendiente sin avanzar	Procesos de despacho, cantidad/antigüedad de EventoSalida pending/sending y proximo_intento_en; conectividad del servicio.
Envío incierto o reconexiones	Conservar clave de idempotencia; revisar lease y aceptación remota antes de reenviar manualmente.
Callback 401	Secreto HMAC, bytes firmados y reloj sincronizado (ventana 300 s).
Match rate_limited	Cuota y espera del proveedor; mostrar estado al usuario y revisar cola externa; no duplicar trabajos para forzar la cuota.
No llegan avisos	ASGI, proxy Upgrade, Redis Channels, sesión/origen/token; probar consulta HTTP de estado.

Síntoma	Revisión / recuperación
Resultado anterior al CV actual	Versiones y hashes; analizar CV vigente, sin sobrescribir historial.
CV heredado público	Bloquear ruta en proxy; respaldar y ejecutar primero manage.py migrar_cv_privados en revisión. --apply mueve archivos y requiere ventana controlada.
PostgreSQL no conecta	Credenciales privadas, red, TLS y DB_ENGINE; comprobar protecciones con tests de configuración.

Proponer alertas por antigüedad de outbox, errores de callback/servicio y espacio disponible. Umbrales y guardia aún deben acordarse. No se configuraron alarmas ni tareas programadas en esta entrega.

Pruebas y calidad

Revisión: 22/10/2026. Complementa la estrategia existente. Estos son comandos y criterios de aceptación, no una declaración de que toda la suite pasó durante la documentación.

Capas y comandos

Desde la raíz, usando el entorno instalado y configuración de pruebas:

```
.venv/bin/python manage.py check
.venv/bin/python manage.py makemigrations --check --dry-run
.venv/bin/python -m unittest ientrabajo.test_database_settings ientrabajo.test_proxy_settings
.venv/bin/python manage.py test usuarios categorias ofertas cursos ia matching moderacion emails
npm run test:js
npm run test:e2e
```

Django crea una base de pruebas; con PostgreSQL la cuenta debe tener permiso para crearla. Revisar la configuración antes de ejecutar: los E2E usan el stack real local y sus fixtures, no la misma base temporal de los unitarios. Los scripts de .cjs adicionales no están todos incluidos en npm run test:js; incorporarlos explícitamente al plan si se validan sus flujos.

El comando npm run test:e2e:live habilita IA real y consumo de cuota; debe ejecutarse deliberadamente. La suite FastAPI se corre en su repositorio; no inventar aquí sus comandos.

Casos críticos y trazabilidad

Requisitos	Casos mínimos	Evidencia principal
RF-01/02/03/14	Registro/email, permisos, empresa pendiente, propiedad de oferta, transiciones, categorías protegidas.	usuarios/tests.py, usuarios/test_integracion.py, ofertas/tests.py, moderacion/tests.py, categorias/tests.py.
RF-05/06	Favoritas únicas, tamaño/formato de CV, versión, descarga ajena/anónima, bloqueo público, migración privada.	Tests de usuarios/ofertas y escenarios E2E.
RF-07/08/13	Callback firmado y repetido, entrega/reintento, reemplazo de CV, resultado tardío, normalización concurrente, canales privados.	ia/tests.py, ia/test_recovery.py, ia/test_profile_skills.py, tests de integración.
RF-09/10/11	Snapshot y hashes, pesos 100%, perfiles vigentes, cancelación, reintento, aislamiento por UUID.	matching/tests.py, usuarios/test_process_limits.py.
RF-12	Cursos según habilidades y brechas, exclusión de recomendaciones no aptas, sin llamada LLM extra.	matching/test_course_recommendations.py, matching/test_course_recommendation_views.py, cursos/test_course_skills.py.

Los archivos se citan como suites relacionadas; comprobar el caso exacto y ampliar cobertura al cambiar el comportamiento. No inferir cobertura total solo por el nombre de una suite.

Dobles, fixtures y datos

En pruebas de integración controladas simular Requests/WebSocket/Redis/LLM según la frontera del caso. Fijar respuestas válidas/erróneas, timeout, rate limit, evento duplicado y callback atrasado. Usar contratos Pydantic reales y fixtures sintéticos sin documentos personales. Las pruebas de precisión de IA son distintas de las que verifican transporte y puntuación.

seed_demo genera datos locales; revisar sus opciones y no ejecutarlo contra producción. Playwright dispone de scripts/e2e_fixture.py y scripts/run_e2e.cjs; los artefactos privados quedan en .test-artifacts/ según la estrategia

existente. Conservar fallos reproducibles y confirmar limpieza de identidades/archivos sintéticos.

Criterio para integrar

Aprobar cuando no hay fallos P0, migraciones y modelos coinciden, contratos son compatibles y pasan las pruebas del área cambiada. Para cambios de integración, ejecutar suite Django, configuración, JS y E2E aplicable. Registrar versión, ambiente, comandos, resultados y omisiones. Un mock o caso omitido no equivale a integración real aprobada.

En .github solo se encontró CODEOWNERS; no hay un workflow de CI de pruebas declarado. Pendiente: automatizar las comprobaciones sin secretos de producción, separar integración con infraestructura de unitarios y publicar reportes. Defectos: registrar RF afectado, reproducción, resultado esperado/actual, severidad y evidencia en el backlog o issue real del repositorio.

Validación de esta entrega

Ver registro de revisión documental, generado al terminar. Los reportes previos de testing en docs/ son evidencia histórica de sus propias fechas; no certifican automáticamente los cambios locales actuales.

Revisión documental del 22/10/2026

Alcance

Se leyó la guía adjunta completa (dos páginas) y se contrastaron modelos, migraciones, rutas, vistas, servicios, configuración, comandos y pruebas declaradas del repositorio local. Se mantuvieron los cambios de aplicación preexistentes. No se consultaron filas de negocio ni se modificaron bases, servicios o credenciales.

Verificaciones realizadas

Verificación	Resultado
Inventario mediante Django _meta	21 entidades propias y 212 campos locales, con relaciones, tablas, null/blank, unicidad y opciones.
Grafo de migraciones con MigrationLoader sin conexión	Hojas registradas en el modelo de datos; incluye migraciones locales de IA 0009 y matching 0007.
.venv/bin/python manage.py check	Sin problemas, 0 incidencias silenciadas.
.venv/bin/python manage.py makemigrations --check --dry-run	No se detectaron cambios pendientes de migración.
git diff --check	Sin errores de espacios en los cambios rastreados.

Entregables

Índice de los siete componentes, requisitos RF-01 a RF-14 con aceptación, arquitectura con diagramas y dos ADR, contratos HTTP/callback/WebSocket, modelo ER completo editable y diccionario de campos, operación, estrategia de calidad y PDF de consulta. Se agregó el índice al README y se documentaron los dos despachadores necesarios para entregar trabajos.

Límites de la revisión

No se ejecutó la suite funcional, E2E, proveedor LLM, carga, restauración de backups ni despliegue. Los comandos operativos productivos quedan como procedimientos propuestos. No se verificó el OpenAPI externo porque el servicio FastAPI no forma parte de esta carpeta. No se aplicaron migraciones. La política de retención y los responsables institucionales siguen pendientes.